Experiment 6 : K-Means and Hierarchical Clustering on Housing Dataset

1. Dataset Source

Dataset used: NJ Housing Prices dataset
Source link: https://huggingface.co/datasets/rajkumar4466/nj-housing-prices-tabular

2. Dataset Description

- The dataset contains tabular data related to housing prices in New Jersey.
- It is used for exploratory data analysis and clustering tasks.

Features

- Numerical features such as price, area, number of rooms, and other housing-related attributes
- Data includes continuous variables suitable for clustering
- Preprocessed using scaling and transformation techniques

Target Variable

- No explicit target variable
- Type: Unsupervised learning (clustering)

Dataset Size

- Contains multiple housing records (exact size depends on dataset version)
- Includes several numerical attributes per record

Key Characteristics

- Real-world tabular dataset
- Contains missing values handled using median imputation
- High-dimensional data reduced using PCA
- Suitable for clustering and pattern discovery

3. Mathematical Formulation of the Algorithm

K-Means Clustering

1. Objective Function:

Minimize within-cluster sum of squares

$$J = \Sigma \Sigma ||x_i - \mu_j||^2$$

Where:

- x_i = Data point
- μ_j = Centroid of cluster j

2. Centroid Update:
$\mu_j = (1/n) \Sigma x_i$

Silhouette Score

$S = (b - a) / \max(a, b)$

Where:

- a = Mean intra-cluster distance
- b = Mean nearest-cluster distance

Hierarchical Clustering

Distance Between Clusters (Ward Method):
Minimizes variance within clusters

$D = \Sigma (x - \mu)^2$

PCA (Dimensionality Reduction)

$Z = XW$

Where:

- X = Original data
- W = Eigenvectors
- Z = Transformed data

4. Algorithm Limitations

- K-Means requires predefined number of clusters
- Sensitive to initialization of centroids
- Assumes spherical cluster shapes
- Hierarchical clustering is computationally expensive for large datasets
- PCA may lead to loss of important information
- Results depend on preprocessing and scaling

5. Methodology / Workflow
Steps Involved

1. Import required libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from datasets import load_dataset

from sklearn.preprocessing import RobustScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score

from scipy.cluster.hierarchy import dendrogram, linkage
```

2. Load dataset

```
dataset = load_dataset("rajkumar4466/nj-housing-prices-tabular")
df = dataset['train'].to_pandas()

df.head()
```

	property_type	zip_code	bedrooms	bathrooms	sqft	lot_size	year_built	price
0	Condo	07022	1	1.0	1387	0.50	2000	838900.0
1	Townhouse	08758	1	1.0	716	0.07	1955	468800.0
2	Single Family	08878	1	1.0	999	0.21	1902	359600.0
3	Condo	07228	1	1.0	777	0.27	2014	345100.0
4	Condo	08294	1	1.0	500	0.14	1970	218900.0

3. Perform data cleaning and preprocessing

```
df.info()
df.describe()
```

```
RangeIndex: 4900 entries, 0 to 4899
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   property_type  4900 non-null   object
 1   zip_code       4900 non-null   object
 2   bedrooms       4900 non-null   int64
 3   bathrooms      4900 non-null   float64
 4   sqft           4900 non-null   int64
 5   lot_size       4900 non-null   float64
 6   year_built     4900 non-null   int64
 7   price          4900 non-null   float64
dtypes: float64(3), int64(3), object(2)
memory usage: 306.4+ KB
```

	bedrooms	bathrooms	sqft	lot_size	year_built	price
count	4900.000000	4900.000000	4900.000000	4900.000000	4900.000000	4.900000e+03
mean	2.417347	1.270816	1407.895102	0.343718	1961.538163	6.152056e+05
std	1.460616	0.513569	877.378578	0.320694	36.499438	3.260664e+05
min	1.000000	1.000000	500.000000	0.050000	1900.000000	1.000000e+05
25%	1.000000	1.000000	807.000000	0.150000	1930.000000	3.879750e+05
50%	2.000000	1.000000	1186.000000	0.250000	1961.000000	5.508500e+05
75%	3.000000	1.500000	1745.000000	0.430000	1993.000000	7.706250e+05
max	6.000000	3.500000	8000.000000	4.300000	2024.000000	3.000000e+06

4. Handle missing values

```
df_numeric = df.select_dtypes(include=[np.number])

# Fill missing values
df_numeric = df_numeric.fillna(df_numeric.median())

df_numeric.head()
```

	bedrooms	bathrooms	sqft	lot_size	year_built	price
0	1	1.0	1387	0.50	2000	838900.0
1	1	1.0	716	0.07	1955	468800.0
2	1	1.0	999	0.21	1902	359600.0
3	1	1.0	777	0.27	2014	345100.0
4	1	1.0	500	0.14	1970	218900.0

5. Apply feature selection (variance threshold)

```
from sklearn.feature_selection import VarianceThreshold

selector = VarianceThreshold(threshold=0.01)
df_var = selector.fit_transform(df_numeric)

df_var = pd.DataFrame(df_var, columns=df_numeric.columns[selector.get_support()])

df_var.head()
```

	bedrooms	bathrooms	sqft	lot_size	year_built	price
0	1.0	1.0	1387.0	0.50	2000.0	838900.0
1	1.0	1.0	716.0	0.07	1955.0	468800.0
2	1.0	1.0	999.0	0.21	1902.0	359600.0
3	1.0	1.0	777.0	0.27	2014.0	345100.0
4	1.0	1.0	500.0	0.14	1970.0	218900.0

6. Remove highly correlated features

```
corr_matrix = df_var.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
to_drop = [column for column in upper.columns if any(upper[column] > 0.9)]
df_reduced = df_var.drop(columns=to_drop)
print("Dropped columns:", to_drop)
df_reduced.head()
```

```
Dropped columns: []
```

	bedrooms	bathrooms	sqft	lot_size	year_built	price
0	1.0	1.0	1387.0	0.50	2000.0	838900.0
1	1.0	1.0	716.0	0.07	1955.0	468800.0
2	1.0	1.0	999.0	0.21	1902.0	359600.0
3	1.0	1.0	777.0	0.27	2014.0	345100.0
4	1.0	1.0	500.0	0.14	1970.0	218900.0

7. Apply log transformation

```
df_log = df_reduced.copy()
for col in df_log.columns:
    df_log[col] = np.log1p(df_log[col])
df_log.head()
```

	bedrooms	bathrooms	sqft	lot_size	year_built	price
0	0.693147	0.693147	7.235619	0.405465	7.601402	13.639848
1	0.693147	0.693147	6.575076	0.067659	7.578657	13.057934
2	0.693147	0.693147	6.907755	0.190620	7.551187	12.792750
3	0.693147	0.693147	6.656727	0.239017	7.608374	12.751592
4	0.693147	0.693147	6.216606	0.131028	7.586296	12.296375

8. Scale data using RobustScaler

```
scaler = RobustScaler()
scaled_data = scaler.fit_transform(df_log)
```

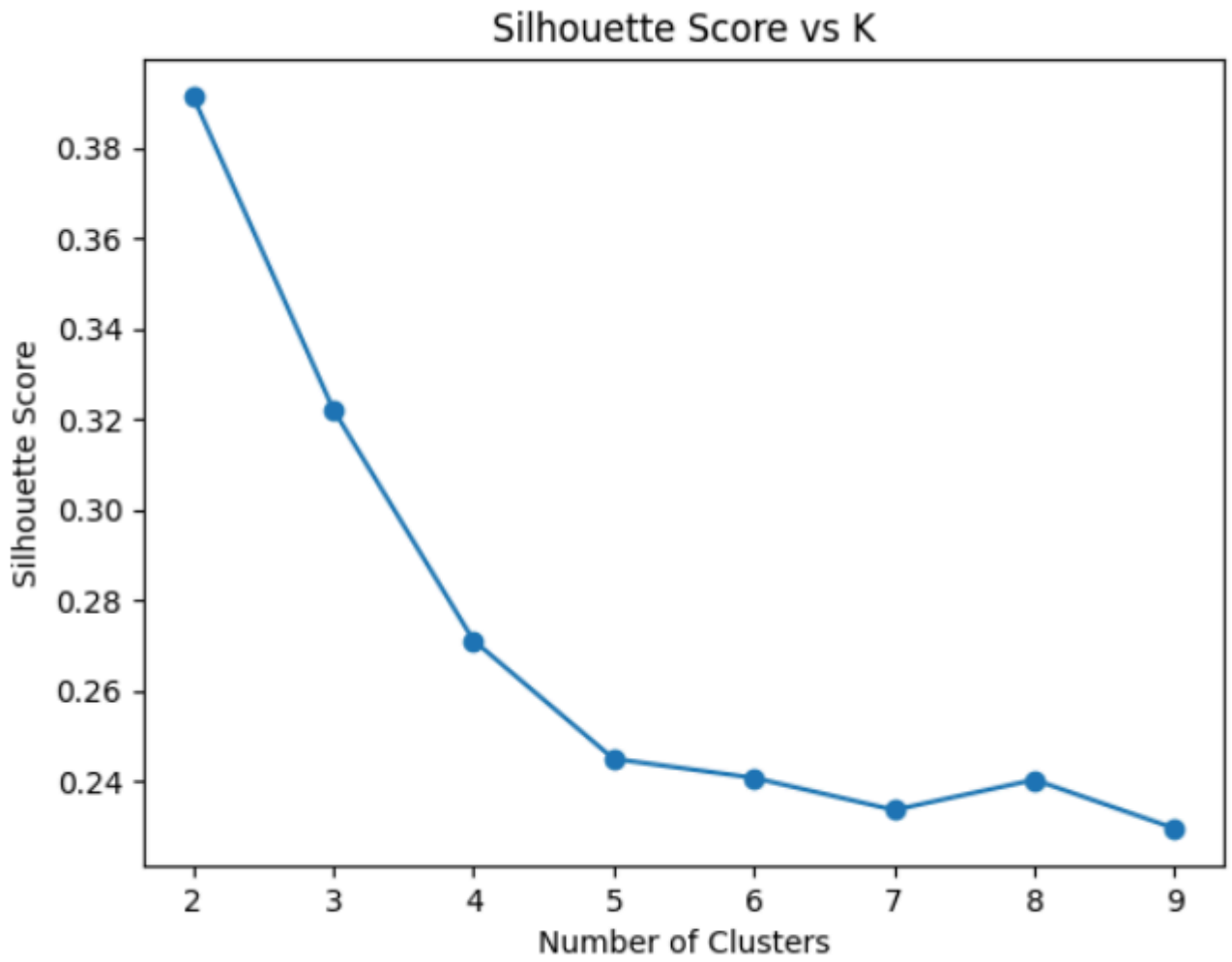
9. Apply PCA for dimensionality reduction

```
pca = PCA(n_components=0.90)  # retain 90% variance
pca_data = pca.fit_transform(scaled_data)

print("Original shape:", scaled_data.shape)
print("Reduced shape:", pca_data.shape)
```

```
Original shape: (4900, 6)
Reduced shape: (4900, 4)
```

10. Perform K-Means clustering

```
scores = []

K_range = range(2, 10)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(pca_data)
    score = silhouette_score(pca_data, labels)
    scores.append(score)

plt.plot(K_range, scores, marker='o')
plt.title("Silhouette Score vs K")
plt.xlabel("Number of Clusters")
plt.ylabel("Silhouette Score")
plt.show()

best_k = K_range[np.argmax(scores)]
print("Best K:", best_k)
```




```
Best K: 2
```

11. Evaluate clusters using silhouette score

```
kmeans = KMeans(n_clusters=best_k, init='k-means++', random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(pca_data)

kmeans_score = silhouette_score(pca_data, kmeans_labels)

print("Optimized K-Means Silhouette Score:", kmeans_score)
```

```
Optimized K-Means Silhouette Score: 0.39140142068785716
```

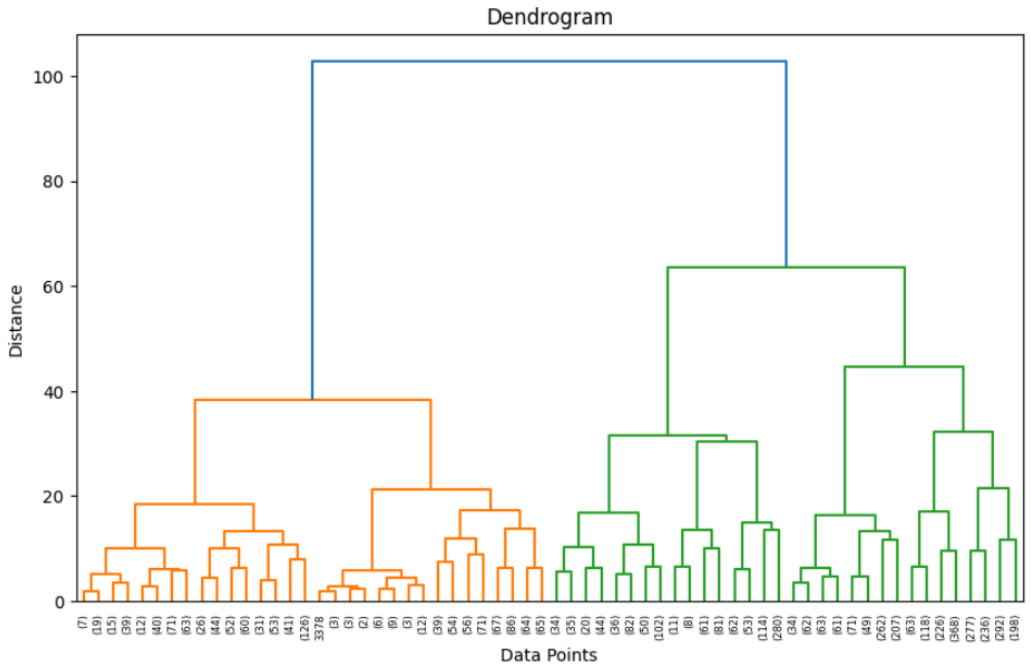
12. Perform hierarchical clustering

```
linkages = ['ward', 'complete', 'average']
best_hc_score = -1
best_linkage = None

for link in linkages:
    hc = AgglomerativeClustering(n_clusters=best_k, linkage=link)
    labels = hc.fit_predict(pca_data)

    score = silhouette_score(pca_data, labels)
    print(f"{link} linkage score:", score)

    if score > best_hc_score:
        best_hc_score = score
        best_linkage = link
        best_hc_labels = labels

print("Best linkage:", best_linkage)
print("Best Hierarchical Score:", best_hc_score)
```

```
ward linkage score: 0.36872466555701777
complete linkage score: 0.32602638601855727
average linkage score: 0.5212014480737747
Best linkage: average
Best Hierarchical Score: 0.5212014480737747
```

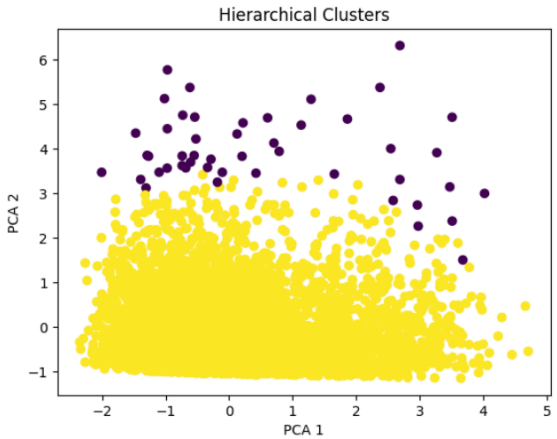
13. Plot dendrogram

```
linked = linkage(pca_data, method='ward')

plt.figure(figsize=(10, 6))
dendrogram(linked, truncate_mode='level', p=5)
plt.title("Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()
```



14. Visualize clusters using PCA

```
pca_2d = PCA(n_components=2)
pca_2d_data = pca_2d.fit_transform(pca_data)

# K-Means plot
plt.scatter(pca_2d_data[:, 0], pca_2d_data[:, 1], c=kmeans_labels)
plt.title("K-Means Clusters")
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
plt.show()

# Hierarchical plot
plt.scatter(pca_2d_data[:, 0], pca_2d_data[:, 1], c=best_hc_labels)
plt.title("Hierarchical Clusters")
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
plt.show()
```




```
print("Final K-Means Score:", kmeans_score)
print("Final Hierarchical Score:", best_hc_score)

Final K-Means Score: 0.39140142068785716
Final Hierarchical Score: 0.5212014480737747
```

Workflow Summary

Raw Data → Preprocessing → Feature Selection → Scaling → PCA → Clustering → Evaluation → Visualization

6. Performance Analysis

- K-Means clustering evaluated using silhouette score
- Best number of clusters selected based on highest score

Interpretation

- Higher silhouette score indicates better-defined clusters
- PCA visualization shows separation between clusters
- K-Means provides faster clustering compared to hierarchical method
- Hierarchical clustering provides better interpretability through dendrogram

7. Hyperparameter Tuning

Parameters Tuned

- Number of clusters (k)
- PCA components (variance retained = 90%)
- Linkage method (ward, complete, average)
- Initialization method (k-means++)

Observations

- Optimal k selected using silhouette score
- PCA reduced dimensionality while preserving variance
- Ward linkage gave better hierarchical clustering results
- Scaling significantly improved clustering performance

Impact

- Improved cluster separation
- Reduced noise and redundancy
- Better visualization and interpretability

Conclusion

The experiment successfully applied K-Means and Hierarchical Clustering to the housing dataset. Data preprocessing and dimensionality reduction played a crucial role in improving clustering performance. The results demonstrated how unsupervised learning can be used to discover patterns and group similar data points effectively.